



第13课时：数组进阶操作

查找·排序·进阶操作

顺序查找、冒泡排序、选择排序及基本操作



为什么需要数组进阶操作？



上一课我们学会了数组的声明、初始化和遍历。但在实际应用中，我们经常需要对数组进行更复杂的处理：

- **查找**：在数组中寻找某个特定元素的位置
- **排序**：将数组元素按从小到大或从大到小排列
- **反转**：将数组元素顺序倒置
- **插入/删除**：在数组的特定位置添加或移除元素

这些操作是算法的基础，也是解决实际问题的重要工具。

顺序查找（线性查找）



顺序查找是最简单的查找方法：从数组的第一个元素开始，逐个比较，直到找到目标或遍历完整整个数组。

适用场景：数据量小或无序数组。

时间复杂度： $O(n)$ ，最坏情况下需要比较 n 次。

顺序查找（线性查找）



顺序查找是最简单的查找方法：从数组的第一个元素开始，逐个比较，直到找到目标或遍历完整整个数组。

```
int main() {  
    int arr[] = {5, 2, 9, 1, 7, 3};  
    int n = 6;    int target = 7;  
    int pos = -1; // -1 表示未找到  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == target) {  
            pos = i;  
            break; // 找到后立即退出  
        }  
    }  
    if (pos != -1)  
        cout << "找到 " << target << ", 下标为 " << pos << endl;  
    else  
        cout << "未找到" << endl;  
    return 0;  
}
```

顺序查找的优化技巧



如果数组是**有序**的（如升序），可以在遍历时提前终止：

```
// 假设数组已升序排列  
for (int i = 0; i < n && arr[i] <= target; i++) {  
    if (arr[i] == target) { pos = i; break; }  
}  
// 当 arr[i] > target 时，后面的元素都更大，无需继续
```

注意：对于无序数组，这种优化不适用。

数组排序的重要性



排序 (Sorting) 是将数组元素按某种规则重新排列的过程。排序后，数据更加有序，便于查找、统计和分析。

常见排序算法：

- 冒泡排序 (Bubble Sort)
- 选择排序 (Selection Sort)
- 插入排序 (Insertion Sort)
- 快速排序 (Quick Sort) 等

本节课我们学习**冒泡排序**和**选择排序**，它们简单易懂，适合入门。



冒泡排序 (Bubble Sort)



思想：重复遍历数组，比较相邻元素，如果顺序错误就交换。每一轮将最大（或最小）元素“冒泡”到数组末尾。

步骤（升序）：

1. 从第一个元素开始，依次比较相邻的两个元素，如果前者大于后者，则交换。
2. 第一轮结束后，最大元素已在末尾。
3. 对剩下的 $n-1$ 个元素重复上述过程。
4. 共进行 $n-1$ 轮。

时间复杂度： $O(n^2)$ 。

冒泡排序代码实现



```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int arr[] = {5, 2, 9, 1, 7, 3};
    int n = 6;

    // 冒泡排序 (升序)
    for (int i = 0; i < n - 1; i++) {           // 轮数
        for (int j = 0; j < n - 1 - i; j++) {   // 每轮比较次数
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);      // 交换
            }
        }
    }

    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    // 输出: 1 2 3 5 7 9
    return 0;
}
```

优化：如果某一轮没有发生交换，说明数组已有序，可提前结束。

冒泡排序过程演示



初始数组: [5, 2, 9, 1, 7, 3]

- **第1轮**: 比较相邻元素, 将最大值 9 移到最后

[5, 2, 9, 1, 7, 3] → [2, 5, 1, 7, 3, 9]

- **第2轮**: 将剩余最大值 7 移到倒数第二

[2, 5, 1, 7, 3, 9] → [2, 1, 5, 3, 7, 9]

- **第3轮**: 将 5 移到正确位置

[2, 1, 5, 3, 7, 9] → [1, 2, 3, 5, 7, 9]

- **第4轮**: 无交换, 已有序, 可提前结束。

选择排序 (Selection Sort)



思想：每一轮从待排序区域中选出最小（或最大）元素，放到已排序区域的末尾。

步骤（升序）：

1. 从第 i 个位置到末尾，找出最小元素的下标。
2. 将最小元素与第 i 个元素交换。
3. i 从 0 递增到 $n-2$ 。

时间复杂度： $O(n^2)$ ，但比冒泡排序的交换次数少。

选择排序代码实现



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int arr[] = {5, 2, 9, 1, 7, 3};
    int n = 6;

    // 选择排序 (升序)
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;           // 假设当前 i 为最小
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;       // 更新最小下标
            }
        }
        if (minIndex != i) {
            swap(arr[i], arr[minIndex]); // 交换
        }
    }

    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    // 输出: 1 2 3 5 7 9
    return 0;
}
```

选择排序过程演示



初始数组: [5, 2, 9, 1, 7, 3]

- **第1轮**: 在 [5, 2, 9, 1, 7, 3] 中找最小元素 1, 与 5 交换 → [1, 2, 9, 5, 7, 3]
- **第2轮**: 在 [2, 9, 5, 7, 3] 中找最小 2, 已在位置 → [1, 2, 9, 5, 7, 3]
- **第3轮**: 在 [9, 5, 7, 3] 中找最小 3, 与 9 交换 → [1, 2, 3, 5, 7, 9]
- **第4轮**: 在 [5, 7, 9] 中找最小 5, 已在位置 → [1, 2, 3, 5, 7, 9]
- 完成。

冒泡排序 vs 选择排序



特性	冒泡排序	选择排序
基本思想	相邻元素比较交换	每轮选出最小元素放到前面
交换次数	较多（可能多次交换）	较少（每轮最多一次交换）
稳定性	稳定（相等元素不交换）	不稳定（可能改变相等元素顺序）
适用场景	小规模数据，教学演示	小规模数据，简单实现
时间复杂度	$O(n^2)$	$O(n^2)$

两者都是入门级排序，实际开发中常用更高效的排序（如快速排序），但理解它们有助于掌握算法思维。



数组元素反转



反转：将数组元素的顺序完全倒置，即第一个与最后一个交换，第二个与倒数第二个交换，以此类推。

实现思路：使用两个指针 `left` 和 `right`，分别指向首尾，交换后 `left++`，`right--`，直到 `left >= right`。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int arr[] = {1, 2, 3, 4, 5, 6};
    int n = 6;

    int left = 0, right = n - 1;
    while (left < right) {
        swap(arr[left], arr[right]);
        left++;
        right--;
    }
}
```

数组元素的插入（概念层面）



插入：在数组的指定位置（下标 pos）插入一个新元素。

难点：数组大小固定，插入后需要移动后续元素以空出位置，且可能导致数组溢出。

步骤（假设数组有足够空间）：

1. 从最后一个元素开始，将每个元素向后移动一位，直到位置 pos。
2. 将新元素放入 arr[pos]。
3. 数组长度增加 1。

```
// 假设 arr 大小足够，当前有效长度为 n
// 在 pos 处插入 value
for (int i = n; i > pos; i--) {
    arr[i] = arr[i - 1];    // 后移
}
arr[pos] = value;
n++;    // 更新长度
```

数组元素的删除（概念层面）



删除：移除数组中指定位置（下标 pos ）的元素。

步骤：

1. 从 $pos+1$ 开始，将每个元素向前移动一位，覆盖 pos 处的元素。
2. 数组长度减 1（多余的元素可忽略）。

```
// 删除下标为 pos 的元素
for (int i = pos; i < n - 1; i++) {
    arr[i] = arr[i + 1];    // 前移
}
n--;    // 更新长度
```

注意：插入和删除在数组中效率较低（ $O(n)$ ），因为需要移动大量元素。对于频繁插入/删除的场景，应考虑链表等数据结构（后续学习）。

插入与删除的完整示例（示意）



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int arr[10] = {1, 2, 3, 4, 5};
    int n = 5;  // 当前有效长度

    // 在位置 2 插入 99
    int pos = 2;
    for (int i = n; i > pos; i--) arr[i] = arr[i-1];
    arr[pos] = 99;
    n++;
    // 现在 arr: 1,2,99,3,4,5

    // 删除位置 3（原3被移除）
    pos = 3;
    for (int i = pos; i < n-1; i++) arr[i] = arr[i+1];
    n--;
    // 现在 arr: 1,2,99,4,5

    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    return 0;
}
```

综合例题：查找、排序与反转结合



题目：输入 n 个整数，先排序（升序），再查找某个目标值，最后将排序后的数组反转输出。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, target;
    cin >> n;
    int arr[100];
    for (int i = 0; i < n; i++) cin >> arr[i];
    cin >> target;
    // 冒泡排序
    for (int i = 0; i < n-1; i++)
        for (int j = 0; j < n-1-i; j++)
            if (arr[j] > arr[j+1]) swap(arr[j], arr[j+1]);

    // 顺序查找（数组已有序，可提前终止）
    int pos = -1;
    for (int i = 0; i < n && arr[i] <= target; i++) {
        if (arr[i] == target) { pos = i; break; }
    }
    cout << "目标位置: " << pos << endl;

    // 反转
    for (int i = 0; i < n/2; i++) swap(arr[i], arr[n-1-i]);
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    return 0;
}
```



错误类型	示例/说明	解决方法
排序时越界	<code>for (j=0; j<n-i; j++)</code> 访问 <code>arr[j+1]</code> 可能越界	确保 <code>j<n-1-i</code> ，控制边界
选择排序中 <code>minIndex</code> 未更新	忘记在找到更小值时更新 <code>minIndex</code>	在循环中正确更新
插入/删除时忘记更新长度	<code>n</code> 未增加或减少，导致后续操作错误	插入后 <code>n++</code> ，删除后 <code>n--</code>
反转时使用错误的循环条件	<code>left <= right</code> 会导致中间元素重复交换	使用 <code>left < right</code>
查找时未处理未找到的情况	直接使用 <code>pos</code> 可能导致输出错误	初始化 <code>pos=-1</code> ，并判断

调试技巧：在排序每轮结束后打印数组，观察变化；插入/删除前后打印数组状态。



知识小结



- **顺序查找**：逐个比较，简单但效率低 ($O(n)$)，适用于无序数组。
- **冒泡排序**：相邻交换，每轮冒出最大元素，稳定， $O(n^2)$ 。
- **选择排序**：每轮选择最小元素放到前面，不稳定， $O(n^2)$ 但交换少。
- **反转**：双指针交换， $O(n)$ 时间， $O(1)$ 空间。
- **插入/删除**：需要移动元素， $O(n)$ 时间，数组长度固定需注意边界。

这些基础操作是后续学习高级算法（如二分查找、快速排序）的前提，务必熟练掌握。

下节课预告



下节课我们将学习**二维数组**，理解多维数据的存储与遍历，并学会处理矩阵相关问题（加法、转置等）。

谢谢大家！



数组进阶操作让数据处理能力更上一层楼！

Q & A